Technical details

Boring stuff

Most of the implementation of this clock is a direct copy of the original hand-rolled javascript version and does not need any particular explanation. The only new improvement on the old clock is an more advanced statistics tracking system.

The new statistics tracking system:

- A flow-chart-esque chart that lists individual events corresponding to their time of occurence rather than a lump-sum of total event duration compared to the old chart.
- Archives old statistics which enables old charts of a particular date to be looked back at
- Specific statistics tracking such as *max_time* and *session to session % change*

This is all entirely attributed to a more advanced state management system in React which enables more complex systems to be built easier.

Fun stuff (the globe clock-face)

Inspired by the globe watch-face on the apple watch, I am implementing a realistic globe render.

Rendering is done with THREE.js

Globe rendering

The core day-night-cycle implementation is based off of:

https://github.com/vasturiano/globe.gl/blob/master/example/day-night-cycle/index.html

Earth image at day, Earth image at night

↓

Show day image on side facing the light, show night image on the opposite

↓

Add an extra factor which for ambient light and simulates luminosity based lighting.

An individual pixel color, $p_i = (r_{\text{i}}, g_{\text{i}}, b_{\text{i}}, a_{\text{i}})$

Image of earth at day, $D = \begin{pmatrix} (r_{00},g_{00},b_{00},a_{00}) & (r_{10},g_{10},b_{10},a_{10}) & \dots & (r_{\text{x}0},g_{\text{x}0},b_{\text{x}0},a_{\text{x}0}) \\ (r_{01},g_{01},b_{01},a_{01}) & (r_{11},g_{11},b_{11},a_{11}) & \dots & (r_{\text{x}1},g_{\text{x}1},b_{\text{x}1},a_{\text{x}1}) \\ \vdots & \vdots & \ddots & \vdots \\ (r_{0\text{y}},g_{0\text{y}},b_{0\text{y}},a_{0\text{y}}) & (r_{1\text{y}},g_{1\text{y}},b_{1\text{y}},a_{1\text{y}}) & \dots & (r_{\text{xy}},g_{\text{xy}},b_{\text{xy}},a_{\text{xy}}) \end{pmatrix}$

Image of earth at night, $N = \begin{pmatrix} (r_{00},g_{00},b_{00},a_{00}) & (r_{10},g_{10},b_{10},a_{10}) & \dots & (r_{\text{x}0},g_{\text{x}0},b_{\text{x}0},a_{\text{x}0}) \\ (r_{01},g_{01},b_{01},a_{01}) & (r_{11},g_{11},b_{11},a_{11}) & \dots & (r_{\text{x}1},g_{\text{x}1},b_{\text{x}1},a_{\text{x}1}) \\ \vdots & \vdots & \ddots & \vdots \\ (r_{0\text{y}},g_{0\text{y}},b_{0\text{y}},a_{0\text{y}}) & (r_{1\text{y}},g_{1\text{y}},b_{1\text{y}},a_{1\text{y}}) & \dots & (r_{\text{xy}},g_{\text{xy}},b_{\text{xy}},a_{\text{xy}}) \end{pmatrix}$

Position of the light source, $L = \begin{pmatrix} x \\ y \\ z \end{pmatrix}$ Normal vector of vertex, $\overrightarrow{\text{VN}} = \begin{pmatrix} x \\ y \\ z \end{pmatrix}$,

Dot procut gives which way is facing the light source; also indicated light intensity.

$$I_0 = L \cdot \overrightarrow{\text{VN}} = \begin{pmatrix} x \\ y \\ z \end{pmatrix} \cdot \begin{pmatrix} x_{\text{vn}} \\ y_{\text{vn}} \\ z_{\text{vn}} \end{pmatrix}$$

Mixing images, shader output color $G = \text{mix}(D, N, \text{B}) = (1 - \text{B})N + \text{B}D$

Blending factor $\text{B} = \mathbb{S}(-0.1, 0.1, I_0)$, smoothstep function $\mathbb{S}$

Luminosity and ambient light $G = \text{mix}(D, N, \text{B}) + \text{K}I_0 + \alpha$

scaling factor for aesehtics : K, ambient Luminosity : α

Sun position calculation

Calculations from NOAA: https://gml.noaa.gov/grad/solcalc/calcdetails.html

Atmosphere

A basic color overlay implementation can be found here:
https://www.youtube.com/watch?v=vM8M4QloVL0

1. **Rayleigh scattering**

$$C \text{ camera position}, L \text{ Light position}$$

$$\overrightarrow{\text{VN}} \text{ vertex normal}, I_0 \text{ Intensity}$$

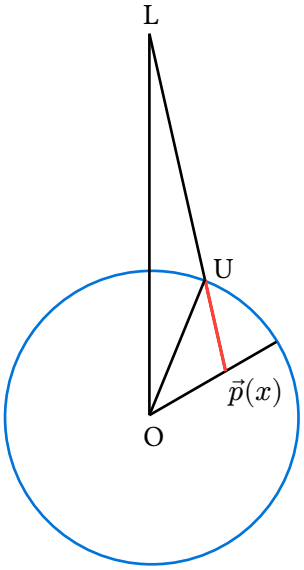
- Position within the atmosphere is modeled by:

$$\vec{p}(x) = x \cdot \overrightarrow{\text{VN}} \qquad R_{\text{earth}} \leq x \leq R_{\text{atmosphere}}$$

given that only the normal vector and the radii of the globe and atmosphere are simple and readily available to obtain.

- Distance to leave the atmosphere based on position:

$\mathbb{F}_i$ is a vector from a point in the atmosphere to a point of interest, light or camera. Where i is the point of interest.



$$\mathbb{F}_L = L - \vec{p}(x) \qquad \mathbb{F}_C = C - \vec{p}(x) \quad \text{e.g for the light and camera}$$

$$\hat{u} = \frac{1}{\|\mathbb{F}_L\|}\mathbb{F}_L \quad \text{is the direction from } \vec{p}(x) \text{ to L}$$

Let U be the intersection point of $\mathbb{F}_i$ on the circle.

Thus triangle $\triangle$ OPU is formed; P = $\overrightarrow{\text{OP}}$

$\overrightarrow{\text{OU}}$ has a length $R_{\text{atmosphere}}$ as the vector ends at the intersection point with the circle.

$$\Rightarrow \overrightarrow{\text{OU}} = \overrightarrow{\text{OP}} + d\hat{u} \qquad \text{where d is the distance from } \vec{p}(x) \text{ to } U$$

$$\Rightarrow R_{\text{atmosphere}} = |\overrightarrow{\text{OU}}| = |\vec{\text{P}} + d\hat{u}| \quad \text{where } \vec{\text{P}} = \overrightarrow{\text{OP}}$$

$$\Rightarrow \left(R_{\text{atmosphere}}\right)^2 = |\vec{\text{P}} + d\hat{u}|^2 = \vec{\text{P}}^2 + 2d\left(\vec{\text{P}} \cdot \hat{u}\right) + d^2$$

$$\Rightarrow x^2 + 2d\left(\vec{\text{P}} \cdot \hat{u}\right) + d^2 - \left(R_{\text{atmosphere}}\right)^2 = 0$$

$$\Rightarrow d^2 + 2d\left(\vec{\text{P}} \cdot \hat{u}\right) + x^2 - \left(R_{\text{atmosphere}}\right)^2 = 0$$

$$A = 1 \quad B = 2\left(\vec{\mathrm{P}} \cdot \hat{u}\right) \quad C = x^2 - \left(R_{\text{atmosphere}}\right)^2$$

$$d = \frac{-2\left(\vec{\mathrm{P}} \cdot \hat{u}\right) \pm \sqrt{\left(-2\left(\vec{\mathrm{P}} \cdot \hat{u}\right)\right)^2 - 4\left(x^2 - \left(R_{\text{atmosphere}}\right)^2\right)}}{2}$$

$$\Rightarrow d = -\left(\vec{\mathrm{P}} \cdot \hat{u}\right) \pm \sqrt{\left(\vec{\mathrm{P}} \cdot \hat{u}\right)^2 - x^2 + \left(R_{\text{atmosphere}}\right)^2}$$

$$\Rightarrow d = -\left(\vec{\mathrm{P}} \cdot \hat{u}\right) + \sqrt{\left(\vec{\mathrm{P}} \cdot \hat{u}\right)^2 - x^2 + \left(R_{\text{atmosphere}}\right)^2}$$